

DSA LEVEL 2

HASHMAP & HASHSET

■ PATTERN: HashMap / HashSet

■ LEVEL OVERVIEW

Fast lookup patterns: existence, frequency, value → index, key → value, complement lookup, grouping, prefix-state counting, and window frequency.

■ TABLE OF CONTENTS

No.	Problem	Pattern	LeetCode
1	Two Sum	HashMap / Complement Lookup	#1
2	Contains Duplicate	HashSet / Existence	#217
3	Valid Anagram	HashMap / Frequency	#242
4	Group Anagrams	HashMap / Canonical Key	#49
5	Majority Element	HashMap / Frequency	#169
6	Intersection of Two Arrays	HashSet / Set Intersection	#349
7	Longest Consecutive Sequence	HashSet / Sequence	#128
8	Subarray Sum Equals K	Prefix Sum + HashMap	#560
9	First Unique Character in a String	HashMap / Frequency + Second Pass	#387
10	Isomorphic Strings	HashMap / Two-Way Mapping	#205
11	Happy Number	HashSet / Seen State	#202
12	Contains Duplicate II	HashMap / Index Distance	#219
13	Find Common Characters	Frequency / Minimum Frequency	#1002
14	Find the Difference of Two Arrays	HashSet / Set Difference	#2215
15	Longest Substring Without Repeating Characters	HashMap + Sliding Window	#3
16	Find All Duplicates in an Array	HashMap / Frequency	#442
17	Count Pairs With Given Sum	HashMap / Complement Counting	■ Absent
18	Longest Subarray With Equal 0s and 1s	Prefix State + HashMap	#525
19	Count Distinct Elements in Every Window of Size K	Sliding Window + HashMap	■ Absent

Code indentation: spaces inside loops, if-statements, functions, and class blocks are preserved exactly. Nested statements visibly begin farther from the opening brace.

C++ colors: **keywords/types** · **directives** · **comments** · **strings** · **STL/built-in functions**.

1. Two Sum

■ Problem 1 ■ LeetCode: #1

■ PATTERN: HashMap / Complement Lookup

■ INPUT An integer array nums and an integer target.

■ OUTPUT Return the two indices whose values add to target.

■ CONSTRAINTS $2 \leq n \leq 10^4$; exactly one valid answer; do not use the same element twice.

■ MAIN IDEA / PATTERN

Store value → index. For nums[i], look for target - nums[i] before inserting the current value.

■ TIME COMPLEXITY

O(n) average

■ SPACE COMPLEXITY

O(n)

■ OPTIMAL C++ CODE

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int, int> mp;

        for (int i = 0; i < nums.size(); i++) {
            int need = target - nums[i];

            if (mp.find(need) != mp.end()) {
                return {mp[need], i};
            }

            mp[nums[i]] = i;
        }

        return {};
    }
};
```

Practice note: Variation: return the pair of values instead of indices.

2. Contains Duplicate

■ Problem 2 ■ LeetCode: #217

■ PATTERN: HashSet / Existence

■ INPUT An integer array nums.

■ OUTPUT Return true if any value appears at least twice; otherwise false.

■ CONSTRAINTS $1 \leq n \leq 10^4$; values may be positive, negative, or repeated.

■ MAIN IDEA / PATTERN

A HashSet answers “Have I seen this value?” Insert each value and stop when it already exists.

■ TIME COMPLEXITY

$O(n)$ average

■ SPACE COMPLEXITY

$O(n)$

■ OPTIMAL C++ CODE

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool containsDuplicate(vector<int>& nums) {
        unordered_set<int> seen;

        for (int x : nums) {
            if (seen.count(x)) {
                return true;
            }

            seen.insert(x);
        }

        return false;
    }
};
```

Practice note: Variation: count how many distinct values are duplicated.

3. Valid Anagram

■ Problem 3 ■ LeetCode: #242

■ PATTERN: HashMap / Frequency

■ INPUT Two strings s and t.

■ OUTPUT Return true if t is an anagram of s.

■ CONSTRAINTS $1 \leq \text{length} \leq 5 \times 10^4$; characters may include lowercase English letters.

■ MAIN IDEA / PATTERN

Count each character in s, then decrement using t. Every final frequency must match.

■ TIME COMPLEXITY

$O(n)$

■ SPACE COMPLEXITY

$O(1)$ for fixed alphabet

■ OPTIMAL C++ CODE

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.size() != t.size()) {
            return false;
        }

        unordered_map<char, int> freq;

        for (char c : s) {
            freq[c]++;
        }

        for (char c : t) {
            freq[c]--;

            if (freq[c] < 0) {
                return false;
            }
        }

        return true;
    }
};
```

Practice note: Variation: replace the HashMap with an array of size 26.

4. Group Anagrams

■ Problem 4 ■ LeetCode: #49

■ PATTERN: HashMap / Canonical Key

■ INPUT An array of strings.

■ OUTPUT Group strings that are anagrams of each other.

■ CONSTRAINTS $1 \leq \text{number of strings} \leq 10^4$; total character count is bounded by the problem.

■ MAIN IDEA / PATTERN

Sort each word to form a canonical key. Equal keys belong to the same group.

■ TIME COMPLEXITY

$O(n \cdot k \log k)$

■ SPACE COMPLEXITY

$O(nk)$

■ OPTIMAL C++ CODE

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        unordered_map<string, vector<string>> mp;

        for (string s : strs) {
            string key = s;
            sort(key.begin(), key.end());

            mp[key].push_back(s);
        }

        vector<vector<string>> ans;

        for (auto& entry : mp) {
            ans.push_back(entry.second);
        }

        return ans;
    }
};
```

Practice note: Variation: use a 26-frequency signature instead of sorting.

5. Majority Element

■ Problem 5 ■ LeetCode: #169

■ PATTERN: HashMap / Frequency

■ INPUT An integer array nums.

■ OUTPUT Return the element that appears more than $n/2$ times.

■ CONSTRAINTS $n \geq 1$; a majority element is guaranteed to exist.

■ MAIN IDEA / PATTERN

Count frequencies and return as soon as a value crosses $n/2$.

■■ TIME COMPLEXITY

$O(n)$ average

■ SPACE COMPLEXITY

$O(n)$

■ OPTIMAL C++ CODE

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int majorityElement(vector<int>& nums) {
        unordered_map<int, int> freq;

        for (int x : nums) {
            freq[x]++;

            if (freq[x] > nums.size() / 2) {
                return x;
            }
        }

        return -1;
    }
};
```

Practice note: Variation: use Boyer-Moore Voting Algorithm for $O(1)$ extra space.

6. Intersection of Two Arrays

■ Problem 6 ■ LeetCode: #349

■ PATTERN: HashSet / Set Intersection

■ INPUT	Two integer arrays nums1 and nums2.
■ OUTPUT	Return unique values present in both arrays.
■ CONSTRAINTS	$1 \leq \text{lengths} \leq 1000$; values may repeat.

■ MAIN IDEA / PATTERN

Put nums1 into a set, then test nums2. A second set prevents duplicate output.

■ TIME COMPLEXITY

$O(n + m)$ average

■ SPACE COMPLEXITY

$O(n + m)$

■ OPTIMAL C++ CODE

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> intersection(vector<int>& nums1, vector<int>& nums2) {
        unordered_set<int> s(nums1.begin(), nums1.end());
        unordered_set<int> ans;

        for (int x : nums2) {
            if (s.count(x)) {
                ans.insert(x);
            }
        }

        return vector<int>(ans.begin(), ans.end());
    }
};
```

Practice note: Variation: return the intersection including duplicates using frequencies.

7. Longest Consecutive Sequence

■ Problem 7 ■ LeetCode: #128

■ PATTERN: HashSet / Sequence

■ INPUT	An unsorted integer array nums.
■ OUTPUT	Return the length of the longest consecutive-element sequence.
■ CONSTRAINTS	$0 \leq n \leq 10^5$; values can be arbitrary integers.

■ MAIN IDEA / PATTERN

Only start a sequence at x when x - 1 is absent. Then extend forward using the set.

■ TIME COMPLEXITY

O(n) average

■ SPACE COMPLEXITY

O(n)

■ OPTIMAL C++ CODE

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int> s(nums.begin(), nums.end());
        int best = 0;

        for (int x : s) {
            if (!s.count(x - 1)) {
                int cur = x;
                int len = 1;

                while (s.count(cur + 1)) {
                    cur++;
                    len++;
                }

                best = max(best, len);
            }
        }

        return best;
    }
};
```

Practice note: Variation: explain why duplicates do not increase the sequence length.

8. Subarray Sum Equals K

■ Problem 8 ■ LeetCode: #560

■ PATTERN: Prefix Sum + HashMap

■ INPUT An integer array nums and integer k.

■ OUTPUT Return the total number of continuous subarrays whose sum equals k.

■ CONSTRAINTS $1 \leq n \leq 2 \times 10^4$; values can be negative, zero, or positive.

■ MAIN IDEA / PATTERN

If current prefix sum is S, a previous prefix S-k gives a subarray sum k. Store prefix-sum frequencies.

■ TIME COMPLEXITY

O(n) average

■ SPACE COMPLEXITY

O(n)

■ OPTIMAL C++ CODE

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int subarraySum(vector<int>& nums, int k) {
        unordered_map<int, int> freq;
        freq[0] = 1;

        int sum = 0;
        int ans = 0;

        for (int x : nums) {
            sum += x;

            if (freq.count(sum - k)) {
                ans += freq[sum - k];
            }

            freq[sum]++;
        }

        return ans;
    }
};
```

Practice note: Variation: find the maximum-length subarray whose sum is k.

9. First Unique Character in a String

■ Problem 9 ■ LeetCode: #387

■ PATTERN: HashMap / Frequency + Second Pass

■ INPUT

A string s.

■ OUTPUT

Return the index of the first non-repeating character, or -1.

■ CONSTRAINTS

$1 \leq \text{length} \leq 10^5$; lowercase English letters.

■ MAIN IDEA / PATTERN

First count frequencies, then scan from left to right and return the first character with frequency 1.

■ TIME COMPLEXITY

$O(n)$

■ SPACE COMPLEXITY

$O(1)$ for fixed alphabet

■ OPTIMAL C++ CODE

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int firstUniqChar(string s) {
        unordered_map<char, int> freq;

        for (char c : s) {
            freq[c]++;
        }

        for (int i = 0; i < s.size(); i++) {
            if (freq[s[i]] == 1) {
                return i;
            }
        }

        return -1;
    }
};
```

Practice note: Variation: return the first unique character instead of its index.

10. Isomorphic Strings

■ Problem 10 ■ LeetCode: #205

■ PATTERN: HashMap / Two-Way Mapping

■ INPUT	Two strings s and t.
■ OUTPUT	Return true if characters in s can be replaced to get t with a one-to-one mapping.
■ CONSTRAINTS	The two strings have equal length; mapping must be consistent and one-to-one.

■ MAIN IDEA / PATTERN

Use $s \rightarrow t$ and $t \rightarrow s$ maps. The reverse map prevents two source characters mapping to the same target.

■ TIME COMPLEXITY

$O(n)$

■ SPACE COMPLEXITY

$O(1)$ for fixed character set

■ OPTIMAL C++ CODE

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool isIsomorphic(string s, string t) {
        unordered_map<char, char> mp1;
        unordered_map<char, char> mp2;

        for (int i = 0; i < s.size(); i++) {
            char a = s[i];
            char b = t[i];

            if (mp1.count(a) && mp1[a] != b) {
                return false;
            }

            if (mp2.count(b) && mp2[b] != a) {
                return false;
            }

            mp1[a] = b;
            mp2[b] = a;
        }

        return true;
    }
};
```

Practice note: Variation: solve with two arrays of size 256.

11. Happy Number

■ Problem 11 ■ LeetCode: #202

■ PATTERN: HashSet / Seen State

■ INPUT A positive integer n.

■ OUTPUT Return true if repeated digit-square sums eventually reach 1.

■ CONSTRAINTS n is positive and repeated states must be handled.

■ MAIN IDEA / PATTERN

Each transformation produces a state. A HashSet detects a repeated state, which means a cycle.

■ TIME COMPLEXITY

$O(\log n)$ per transformation; bounded cycle

■ SPACE COMPLEXITY

$O(\log n)$

■ OPTIMAL C++ CODE

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool isHappy(int n) {
        unordered_set<int> seen;

        while (n != 1 && !seen.count(n)) {
            seen.insert(n);

            int sum = 0;

            while (n > 0) {
                int digit = n % 10;
                sum += digit * digit;
                n /= 10;
            }

            n = sum;
        }

        return n == 1;
    }
};
```

Practice note: Variation: detect the cycle with Floyd's cycle detection.

12. Contains Duplicate II

■ Problem 12 ■ LeetCode: #219

■ PATTERN: HashMap / Index Distance

■ INPUT	An integer array nums and integer k.
■ OUTPUT	Return true if two equal values have indices within distance k.
■ CONSTRAINTS	$1 \leq n$; $k \geq 0$.

■ MAIN IDEA / PATTERN

Store the latest index for each value. If the current index minus the latest index is $\leq k$, the condition is satisfied.

■ TIME COMPLEXITY

$O(n)$ average

■ SPACE COMPLEXITY

$O(n)$

■ OPTIMAL C++ CODE

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool containsNearbyDuplicate(vector<int>& nums, int k) {
        unordered_map<int, int> last;

        for (int i = 0; i < nums.size(); i++) {
            if (last.count(nums[i]) &&
                i - last[nums[i]] <= k) {
                return true;
            }

            last[nums[i]] = i;
        }

        return false;
    }
};
```

Practice note: Variation: use a sliding HashSet containing only the last k values.

13. Find Common Characters

■ Problem 13 ■ LeetCode: #1002

■ PATTERN: Frequency / Minimum Frequency

■ INPUT	An array of lowercase strings.
■ OUTPUT	Return all characters that appear in every string, including repeated occurrences.
■ CONSTRAINTS	Each word contains lowercase English letters.

■ MAIN IDEA / PATTERN

For every word, compute its 26 frequencies and keep the minimum frequency seen for each character.

■ TIME COMPLEXITY

O(total characters)

■ SPACE COMPLEXITY

O(1) for fixed alphabet

■ OPTIMAL C++ CODE

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<string> commonChars(vector<string>& words) {
        vector<int> common(26, 1000);

        for (string word : words) {
            vector<int> freq(26, 0);

            for (char c : word) {
                freq[c - 'a']++;
            }

            for (int i = 0; i < 26; i++) {
                common[i] = min(common[i], freq[i]);
            }
        }

        vector<string> ans;

        for (int i = 0; i < 26; i++) {
            while (common[i]--) {
                ans.push_back(string(1, 'a' + i));
            }
        }

        return ans;
    }
};
```

Practice note: Variation: return only the distinct common characters.

14. Find the Difference of Two Arrays

■ Problem 14 ■ LeetCode: #2215

■ PATTERN: HashSet / Set Difference

■ INPUT Two integer arrays nums1 and nums2.

■ OUTPUT Return unique values present in nums1 but not nums2, and vice versa.

■ CONSTRAINTS $1 \leq \text{lengths} \leq 1000$; values may repeat.

■ MAIN IDEA / PATTERN

Build two sets. Values in one set that are absent from the other belong to that side's difference.

■ TIME COMPLEXITY

$O(n + m)$ average

■ SPACE COMPLEXITY

$O(n + m)$

■ OPTIMAL C++ CODE

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<vector<int>> findDifference(
        vector<int>& nums1,
        vector<int>& nums2
    ) {
        unordered_set<int> a(nums1.begin(), nums1.end());
        unordered_set<int> b(nums2.begin(), nums2.end());

        vector<int> onlyA;
        vector<int> onlyB;

        for (int x : a) {
            if (!b.count(x)) {
                onlyA.push_back(x);
            }
        }

        for (int x : b) {
            if (!a.count(x)) {
                onlyB.push_back(x);
            }
        }

        return {onlyA, onlyB};
    }
};
```

Practice note: Variation: return frequencies of values unique to each side.

15. Longest Substring Without Repeating Characters

■ Problem 15 ■ LeetCode: #3

■ PATTERN: HashMap + Sliding Window

■ INPUT

A string s.

■ OUTPUT

Return the length of the longest substring without repeated characters.

■ CONSTRAINTS

$0 \leq \text{length} \leq 5 \times 10^4$.

■ MAIN IDEA / PATTERN

Keep a left boundary and the latest index of every character. When a repeat appears, jump left past its previous index.

■ TIME COMPLEXITY

$O(n)$

■ SPACE COMPLEXITY

$O(k)$

■ OPTIMAL C++ CODE

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        unordered_map<char, int> last;
        int left = 0;
        int best = 0;

        for (int right = 0; right < s.size(); right++) {
            char c = s[right];

            if (last.count(c)) {
                left = max(left, last[c] + 1);
            }

            last[c] = right;
            best = max(best, right - left + 1);
        }

        return best;
    }
};
```

Practice note: Variation: return the actual longest substring.

16. Find All Duplicates in an Array

■ Problem 16 ■ LeetCode: #442

■ PATTERN: HashMap / Frequency

■ INPUT An integer array nums.

■ OUTPUT Return all values that appear exactly twice.

■ CONSTRAINTS $1 \leq n \leq 10^4$; each value is in the range $1..n$; each value appears once or twice.

■ MAIN IDEA / PATTERN

Count frequencies with a HashMap, then collect entries with frequency 2.

■ TIME COMPLEXITY

$O(n)$ average

■ SPACE COMPLEXITY

$O(n)$

■ OPTIMAL C++ CODE

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    vector<int> findDuplicates(vector<int>& nums) {
        unordered_map<int, int> freq;
        vector<int> ans;

        for (int x : nums) {
            freq[x]++;
        }

        for (auto& entry : freq) {
            if (entry.second == 2) {
                ans.push_back(entry.first);
            }
        }

        return ans;
    }
};
```

Practice note: Variation: use sign marking for $O(1)$ extra space under the given value constraints.

17. Count Pairs With Given Sum

■ Problem 17 ■ LeetCode: ■ Absent

■ PATTERN: HashMap / Complement Counting

■ INPUT An integer array nums and target k.

■ OUTPUT Return the number of index pairs (i, j), $i < j$, whose values sum to k.

■ CONSTRAINTS Array length is finite; pair counting includes different indices even when values are equal.

■ MAIN IDEA / PATTERN

For each x, add the number of previous occurrences of $k-x$, then insert x.

■ TIME COMPLEXITY

$O(n)$ average

■ SPACE COMPLEXITY

$O(n)$

■ OPTIMAL C++ CODE

```
#include <bits/stdc++.h>
using namespace std;

int countPairs(vector<int>& nums, int k) {
    unordered_map<int, int> freq;
    int ans = 0;

    for (int x : nums) {
        int need = k - x;

        if (freq.count(need)) {
            ans += freq[need];
        }

        freq[x]++;
    }

    return ans;
}
```

Practice note: No exact LeetCode problem used here. Variation: count only unique value-pairs.

18. Longest Subarray With Equal 0s and 1s

■ Problem 18 ■ LeetCode: #525

■ PATTERN: Prefix State + HashMap

■ INPUT A binary array nums.

■ OUTPUT Return the maximum length of a contiguous subarray containing equal numbers of 0 and 1.

■ CONSTRAINTS nums contains only 0 and 1.

■ MAIN IDEA / PATTERN

Treat 0 as -1 and 1 as +1. Equal prefix sums at two indices mean the subarray between them has equal 0s and 1s. Store the first index for each sum.

■ TIME COMPLEXITY

O(n)

■ SPACE COMPLEXITY

O(n)

■ OPTIMAL C++ CODE

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int findMaxLength(vector<int>& nums) {
        unordered_map<int, int> first;
        first[0] = -1;

        int sum = 0;
        int best = 0;

        for (int i = 0; i < nums.size(); i++) {
            sum += (nums[i] == 1 ? 1 : -1);

            if (first.count(sum)) {
                best = max(best, i - first[sum]);
            } else {
                first[sum] = i;
            }
        }

        return best;
    }
};
```

Practice note: Variation: change the prefix-state definition for other equal-count conditions.

19. Count Distinct Elements in Every Window of Size K

■ Problem 19 ■ LeetCode: ■ Absent

■ PATTERN: Sliding Window + HashMap

■ INPUT	An integer array nums and window size k.
■ OUTPUT	Return the number of distinct values in each window of length k.
■ CONSTRAINTS	$1 \leq k \leq n$; array values may repeat.

■ MAIN IDEA / PATTERN

Maintain a frequency map for the current window. Decrement the outgoing value and erase it when its frequency reaches zero.

■ TIME COMPLEXITY

$O(n)$ average

■ SPACE COMPLEXITY

$O(k)$

■ OPTIMAL C++ CODE

```
#include <bits/stdc++.h>
using namespace std;

vector<int> countDistinct(vector<int>& nums, int k) {
    unordered_map<int, int> freq;
    vector<int> ans;

    for (int i = 0; i < k; i++) {
        freq[nums[i]]++;
    }

    ans.push_back(freq.size());

    for (int i = k; i < nums.size(); i++) {
        freq[nums[i - k]]--;

        if (freq[nums[i - k]] == 0) {
            freq.erase(nums[i - k]);
        }

        freq[nums[i]]++;
        ans.push_back(freq.size());
    }

    return ans;
}
```

Practice note: No exact LeetCode problem used here. Variation: find the most frequent value in every window.

LEVEL 2 MASTER CHECKLIST

- Recognize existence, frequency, index, complement, grouping, prefix-state, and window-frequency HashMap/HashSet patterns.
- Explain exactly what information is stored in the key/value pair.
- Recognize when a HashSet is sufficient and when a HashMap is required.
- Recognize prefix sum + HashMap for subarray counting.
- Recognize when a two-way mapping is required.
- State average lookup complexity and extra-space complexity correctly.